

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I __S.Jotheeswaran(192424155)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.Jotheeswaran

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
		factors involved			
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Scenario Based Assignment — Solutions

Artificial Intelligence: Search, Optimization, CSP and Game Playing

Scenario 1: Autonomous Delivery Drone (Search Strategy Selection)

A government deploys AI-powered drones to deliver medicines in flood-affected regions. The environment is highly dynamic — roads may be inaccessible and weather conditions may change. The drone must reach a location as quickly as possible, using either Greedy Best-First Search or A* Search.

i. Greedy Best-First Search vs A* Search — Strengths and Weaknesses

[3 Marks]

Greedy Best-First Search (GBFS) expands the node that appears closest to the goal, using only the heuristic function $h(n)$ to guide the search, while ignoring the cost already incurred, $g(n)$.

- Strength: GBFS is fast and memory-efficient because it aggressively moves toward the goal, which is valuable when the drone needs a quick decision during an emergency.
- Weakness: GBFS is neither complete nor optimal in general — it can get trapped in loops or dead-ends (e.g., a flooded road that looks close to the destination) and may return a much longer path than necessary.

A* Search evaluates nodes using $f(n) = g(n) + h(n)$, combining the actual cost so far with the estimated cost to the goal.

- Strength: A* is complete and optimal when the heuristic is admissible (never overestimates) and consistent, making it reliable for finding the true shortest/safest delivery route.
- Weakness: A* consumes more memory and computation because it must retain and re-evaluate a larger frontier of nodes, which can be a problem for a drone with limited onboard compute and battery.

Given that human lives depend on timely and correct delivery, A* is generally the safer choice because it guarantees an optimal path when a good heuristic is available, whereas GBFS trades correctness for raw speed.

ii. Why A* Handles Real-World Uncertainty Better Than Basic Heuristics

[3 Marks]

Basic heuristic-only approaches (like GBFS) commit to a path purely on 'how good it looks,' which becomes unreliable when the environment changes unpredictably (blocked roads, shifting weather).

- A* keeps track of the accumulated real cost $g(n)$, so if a previously promising route turns out to be blocked or costlier, A* can revise its priority and switch to a better-costed alternative rather than blindly committing.
- Because A* explores in order of total estimated cost $f(n)$, it naturally balances exploration and exploitation — it does not ignore already-invested cost the way GBFS does.
- In practice, A* is combined with periodic re-planning (or Replanning A* / D* Lite variants) so that when new sensor data reveals a blocked route, the search can efficiently repair the plan instead of restarting from scratch — making it much better suited to dynamic, uncertain terrain than a purely heuristic-driven method.

iii. Analysing Heuristic Accuracy on Both Algorithms

[3 Marks]

Heuristic accuracy affects the two algorithms very differently.

- For GBFS: since search is driven purely by $h(n)$, an inaccurate or misleading heuristic (e.g., straight-line distance across a flooded river that has no bridge) can send the drone into long detours or infinite loops — GBFS has no mechanism to 'notice' that a path is actually expensive until it has already committed.

- For A*: an admissible and consistent heuristic guarantees optimality. If the heuristic underestimates too aggressively (close to 0), A* behaves like uniform-cost search — correct but slow, exploring many more nodes than necessary.
- If the heuristic overestimates (inadmissible), A* may return a suboptimal path, exactly like GBFS — so heuristic accuracy is the single biggest factor separating A*'s theoretical optimality from practical performance.

Conclusion: an accurate, admissible heuristic (e.g., real-time terrain-aware distance estimates factoring in flood zones) is essential; poor heuristics degrade A* toward brute-force search and make GBFS actively unsafe.

iv. Recommended Search Algorithm — Justification

[6 Marks]

Recommendation: Use A* Search, supported by a well-engineered, admissible, and (if possible) consistent heuristic (e.g., a weighted combination of geographic distance and a real-time flood/obstacle risk map).

- Optimality with safety guarantees: Medicine delivery is safety-critical; A*'s optimality guarantee (given an admissible heuristic) ensures the drone takes the genuinely best available route, not just the one that 'looks' best.
- Completeness: A* is guaranteed to find a solution if one exists, which matters when road/air corridors are unpredictable.
- Adaptability: A* can be embedded in an online/replanning framework, allowing it to update its plan as new information about blocked routes or weather arrives, addressing the dynamic nature of flood zones.
- Practical mitigation for A*'s memory cost: Use a bounded/anytime variant (e.g., Weighted A*, IDA*, or Real-Time A*) to control memory and give a 'good enough' answer quickly, then refine — combining GBFS-like speed with A*-like reliability.

Overall justification: The cost of a wrong or delayed decision (in GBFS) is far more dangerous than the extra computation A* requires, and modern hardware on delivery drones can typically support A*'s memory needs. A* (or a bounded/anytime variant) is therefore the recommended strategy.

Scenario 2: Smart City Traffic Signal Optimization (Local Search)

A smart city implements an AI system to optimize traffic signal timing across hundreds of intersections. The objective is to minimize total waiting time, fuel consumption, and traffic congestion, with traffic patterns changing dynamically and the search space being extremely large — so the system must find good local solutions.

i. Formulating Traffic Signal Timing as an Optimization Problem

[3 Marks]

This is best modeled as a local search / optimization problem rather than a path-finding problem, since we are not searching for a sequence of actions to a goal state but rather searching for the best configuration (assignment of signal timings) that optimizes an objective function.

- State: a complete assignment of green/red light durations (and phase offsets) to every traffic signal in the city at a given time window.
- Objective/Cost function: a weighted combination of average vehicle waiting time, total fuel consumption, and congestion level (e.g., queue length) — the system searches for the state that minimizes this function.
- Neighborhood/Successor function: small changes to one or more signals' timings (e.g., +5 seconds green time at intersection X) define neighboring states.
- Because the search space (all possible timing combinations across hundreds of intersections) is astronomically large, an exhaustive/systematic search is computationally infeasible — this is precisely the situation where local search algorithms are appropriate: we do not need the guaranteed optimal path, just a state with a very good (low) objective value, found efficiently.

ii. Local Search Techniques and Their Limitations

[3 Marks]

Hill Climbing repeatedly moves to the neighboring state with the best improvement in the objective function.

- Limitation: it easily gets stuck at local optima (e.g., a signal-timing configuration that cannot be improved by any single small change but is far from the city-wide best), on plateaus (regions of equal cost, common when many signals barely affect the objective), and on ridges — all of which are very likely in a search space this large and complex.

Simulated Annealing allows occasional 'bad' moves (that increase cost) with a probability that decreases over time (controlled by a temperature schedule), letting the search escape local optima.

- Limitation: performance is highly sensitive to the cooling schedule — cooling too fast behaves like hill climbing and gets stuck; cooling too slowly wastes computation time, which is problematic for a system that needs to react to changing traffic patterns in near real time.

Genetic Algorithms maintain a population of candidate timing-plans, and use selection, crossover, and mutation to evolve better solutions over generations.

- Limitation: GAs are computationally expensive per iteration (evaluating many candidates), require careful design of the representation/crossover operators for timing plans, and can suffer from premature convergence if diversity in the population is lost too early.

iii. Best Approach and Justification for Scalability and Adaptability

[3 Marks]

Recommended approach: Simulated Annealing for continuous, city-wide fine-tuning, combined with a Genetic Algorithm run periodically (e.g., off-peak hours) to discover strong baseline timing plans, and Hill Climbing (or steepest-ascent variants) for fast local corrections at individual intersections.

- Simulated annealing balances exploration and exploitation and its escape-from-local-optima property is critical given the enormous, rugged search space across hundreds of intersections.
- It is more lightweight per iteration than a GA, making it better suited to near real-time adaptation as traffic patterns change throughout the day.
- A GA can be used in a slower, background loop to periodically re-discover globally strong solutions (useful since traffic patterns shift by day of week, season, events, etc.), whose output can seed/re-initialize the SA process.
- This hybrid approach directly addresses scalability (decomposing into local corrections rather than one huge combinatorial search) and adaptability (SA's continuous fine-tuning responds to real-time changes, while periodic GA runs prevent long-term drift into poor configurations).

Scenario 3: Autonomous Mars Rover (Online Search under Partial Observability)

An autonomous rover is deployed on Mars to explore unknown terrain. It must navigate safely to collect samples, avoid obstacles, and make decisions without complete knowledge of the map — an online search agent operating in a partially observable, dynamic environment.

i. Why This Requires Online Search Instead of Offline Search

[3 Marks]

Offline search algorithms (like standard A* or BFS) require complete, prior knowledge of the state space — they compute an entire solution path before execution begins, assuming the environment is static and fully known.

- The rover cannot use offline search because Mars terrain is not known in advance in full detail — the rover only discovers obstacles, craters, and hazards as its sensors reach them (partial observability).
- The environment may also change (dust storms, shifting rocks), so a plan computed entirely offline could quickly become invalid.
- Online search interleaves computation and action: the rover acts, observes the actual outcome/new local terrain, and then re-plans based on updated knowledge — essential when the agent's information is incomplete and communication delays (~4–24 minutes one-way to Earth) make real-time human intervention impossible.

ii. Challenges of Partial Observability and Dynamic Environments

[3 Marks]

- Incomplete map knowledge: the rover may commit to a route that later turns out to lead to a dead end (e.g., an impassable cliff), forcing costly backtracking — unlike offline search, this backtracking has a real physical/time/energy cost, not just a computational one.
- Sensor noise and limited range: the rover can only observe terrain within sensor range, so its local model of the world is always incomplete, requiring frequent re-evaluation of its plan as new areas come into view.
- Irreversible actions: some actions (e.g., driving into a loose sand trap) may be irreversible or very costly to undo, so the rover must be cautious/conservative — pure trial-and-error online search (like plain online DFS) can be dangerous.
- Dynamic changes: weather (dust storms) and terrain shifts mean that even previously mapped areas can become invalid, so the rover cannot fully trust its own history.
- Resource constraints: limited battery/energy and communication bandwidth restrict how much exploration and re-planning the rover can perform, requiring an efficient balance between exploration (mapping unknown areas) and exploitation (reaching the sample-collection goal).

iii. Describing How the Rover Builds and Updates Its Internal Model

[3 Marks]

The rover maintains an internal representation (map) of the environment that is incrementally built and refined:

1. Initial state: the rover starts with minimal or no map information, only its own position and the general goal direction/coordinates.
2. Sensing: onboard sensors (cameras, LIDAR) scan the immediate surroundings and detect obstacles, slopes, and hazards within range.
3. Model update: newly sensed information is merged into the rover's internal map (e.g., an occupancy grid), marking cells as free, occupied, or unknown.
4. Local re-planning: using this updated model, the rover re-computes the best next action or short path segment (often using an online variant of A*, such as LRTA* — Learning Real-Time A* — which updates heuristic estimates for states it has visited).
5. Execution and feedback: the rover executes the next action, observes the actual result (did it succeed as expected?), and loops back to sensing — continuously tightening the gap between its internal model and reality.

Algorithms such as LRTA* are particularly suited here because they combine acting with learning: each time the rover visits a state, it updates that state's heuristic estimate based on real experience, so the rover's internal model gets progressively more accurate the more it explores — a good fit for one-time or repeated exploration of unknown terrain.

iv. Comparing Online Search with Uninformed and Informed Search Strategies

[3 Marks]

Aspect	Uninformed Search (e.g., BFS/DFS)	Informed Offline Search (e.g., A*)	Online Search (e.g., Online DFS, LRTA*)
Prior knowledge of environment	Assumes full knowledge	Assumes full knowledge + heuristic	Assumes little/no prior knowledge
When plan is computed	Entirely before acting	Entirely before acting	Interleaved with acting (act, observe, replan)
Handles dynamic/unknown terrain	No	No	Yes — designed for it
Cost of mistakes	Purely computational (revisit in memory)	Purely computational	Physical — wasted time/energy/risk to the rover
Efficiency	Can be very slow (large branching factor)	Efficient with a good heuristic	Efficient over repeated attempts as the model improves (e.g., LRTA*)

In short: uninformed search is complete but blind and inefficient; informed offline search (A*) is efficient but requires full prior map knowledge the rover does not have; online search sacrifices the guarantee of finding the globally optimal path in one shot, but is the only category of the three that is actually feasible when the environment is unknown and dynamic, since it lets the rover learn and adapt as it explores.

v. Justifying the Most Suitable Approach (Considering Uncertainty, Adaptability, Safety)

[3 Marks]

Recommended approach: An online, informed search strategy — specifically LRTA* (Learning Real-Time A*) or a similar real-time heuristic search — layered with conservative safety checks before executing irreversible actions.

- **Uncertainty:** LRTA* is explicitly designed for agents that must act without complete knowledge; it updates its understanding of the world (heuristic values) as it senses new terrain, directly addressing partial observability.
- **Adaptability:** because it interleaves planning and acting, it naturally adapts to dynamic changes (e.g., a dust storm revealing/hiding terrain) without needing to restart a full offline plan.
- **Safety:** the rover should combine LRTA* with a look-ahead/verification step (checking sensor data before committing to potentially irreversible moves such as descending a slope), avoiding the pitfall of purely reactive search in high-risk terrain.
- **Resource efficiency:** LRTA*'s bounded look-ahead avoids the heavy memory/computation cost of full A*, which matters given the rover's limited onboard power — making it the best fit among available approaches for the mission's constraints.

Scenario 4: University Exam Timetabling (Constraint Satisfaction Problem)

A university is designing an AI system to automatically generate exam timetables. The system must handle multiple constraints: limited number of exam halls, certain subjects that cannot be scheduled at the same time, faculty availability constraints for some students, and increasing complexity as the number of courses and students grows.

i. Formulating the Problem as a Constraint Satisfaction Problem (CSP)

[4 Marks]

A CSP is formally defined by a set of variables, their domains, and the constraints relating them; this scenario maps naturally onto that structure.

Variables

- One variable for each exam/course to be scheduled, e.g., Exam₁, Exam₂, ..., Exam_n (representing subjects such as Mathematics, Physics, Computer Science, etc.).

Domains

- For each exam variable, the domain is the set of possible (time-slot, hall) pairs it could be assigned — e.g., {Day1-Slot1-HallA, Day1-Slot1-HallB, Day1-Slot2-HallA, ...} — restricted to available halls and permitted time slots.

Constraints

- Unary constraints: an exam can only be assigned a hall large enough for its enrolled students, or a time slot within the allowed exam period.
- Binary constraints: two subjects that cannot be scheduled at the same time (e.g., because many students take both) must be assigned different time slots — a 'not-equal' constraint between their time-slot values.
- Resource constraints: no two exams can be assigned the same hall at the same time slot (limited number of exam halls).
- Global constraints: a student cannot have two exams scheduled simultaneously (all exams containing overlapping student enrollments must occupy different slots) — this is essentially a graph-coloring-style constraint across all exams sharing students.
- Faculty availability constraints: exams requiring a specific invigilator/faculty member must be scheduled only within that faculty member's available time slots.

Goal: find a complete assignment of (time-slot, hall) pairs to every exam such that every constraint above is satisfied — a valid, conflict-free timetable.

ii. Explaining How Backtracking Search Works in This Scenario

[3 Marks]

Backtracking search incrementally builds a solution by assigning values to variables one at a time, checking constraints after each assignment, and undoing ('backtracking') when a dead end is reached.

6. Select an unassigned exam (variable) — e.g., using the Minimum Remaining Values heuristic to pick the exam with the fewest valid (time-slot, hall) options left.
7. Try assigning it a value (a specific time-slot + hall) from its domain.
8. Check consistency: verify this assignment does not violate any constraint with previously assigned exams (no student clash, no hall double-booking, no forbidden same-day pairing, faculty available).
9. If consistent, recursively move on to the next unassigned exam.
10. If no value works for the current exam (all lead to conflicts), backtrack: undo the previous exam's assignment and try its next alternative value.
11. Repeat until either every exam has a valid assignment (solution found) or all possibilities are exhausted (no valid timetable exists with current constraints/resources).

This systematic 'assign, check, backtrack-on-failure' process guarantees that if a valid timetable exists, backtracking search will eventually find it, since it explores the search tree of partial assignments completely (with pruning) rather than blindly guessing.

iii. Discussing Constraint Propagation Techniques to Improve Efficiency

[3 Marks]

Plain backtracking can be slow because it only detects a conflict after making a 'bad' assignment. Constraint propagation techniques detect conflicts earlier by reducing variable domains proactively.

- Forward Checking: whenever an exam is assigned a (time-slot, hall) value, immediately remove that value (and other now-invalid combinations) from the domains of all still-unassigned exams connected to it by a constraint. If any domain becomes empty, backtrack immediately rather than continuing further down a doomed path.
- Arc Consistency (e.g., the AC-3 algorithm): goes further than forward checking by ensuring that for every pair of constrained exams (an 'arc'), every remaining value in one exam's domain has at least one compatible value in the other's domain — repeatedly pruning domains until no further reduction is possible, which can detect conflicts several steps ahead rather than only immediate ones.
- Both techniques significantly cut down the search tree explored by backtracking, which matters greatly as the number of courses/students grows, since the raw search space of exam-hall-slot combinations grows combinatorially.
- Additional heuristics such as Minimum Remaining Values (choose the most constrained exam first) and Least Constraining Value (try the hall/slot that rules out the fewest options for other exams first) further improve efficiency when combined with forward checking/arc consistency.

iv. Analyzing Real-World Challenges and Best Strategy for Scalability

[3 Marks]

Real-world challenges:

- Combinatorial explosion: as the number of courses and students grows, the number of possible (time-slot, hall) combinations and inter-exam constraints grows very rapidly, making naive backtracking impractically slow.
- Over-constrained problems: sometimes no perfect solution exists (e.g., too many overlapping-student subjects for too few slots), requiring the system to relax constraints (soft constraints with penalties) rather than fail outright.
- Dynamic changes: late course registrations, faculty leave, or hall unavailability may require re-solving or locally repairing an already-published timetable without disrupting it entirely.
- Fairness and soft preferences: students may prefer gaps between exams, and faculty may prefer specific slots — these are not hard constraints but should be optimized where possible, turning the problem into a constrained optimization problem, not a pure CSP.

Best strategy for scalability: combine backtracking search with constraint propagation (forward checking + arc consistency) and strong variable/value ordering heuristics (MRV, degree heuristic, least-constraining value) as the core solver, and additionally apply local search / min-conflicts repair for very large instances — min-conflicts starts from a complete (possibly conflicting) assignment and iteratively re-assigns the exam involved in the most conflicts to the value that minimizes conflicts, which scales far better than exhaustive backtracking on large, real-world-sized timetabling problems, and can also efficiently repair a timetable after last-minute changes.

Scenario 5: Strategic Game AI (Real-Time Decision Making using Minimax / Alpha-Beta Pruning)

A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, making decisions within strict time limits. The game involves complex, large possible moves, making decision-making extremely computationally expensive.

i. Explaining How the Minimax Algorithm Models This Problem

[3 Marks]

Minimax models the game as a two-player, zero-sum adversarial search over a game tree, where one player (the AI, the maximizer) tries to maximize its outcome/utility, and the opponent (the human, the minimizer) tries to minimize it, assuming the opponent also plays optimally.

- The game tree's nodes represent game states, and edges represent legal moves; at MAX levels the AI picks the move leading to the highest-valued resulting state, and at MIN levels it assumes the opponent picks the move leading to the lowest-valued state (worst case for the AI).
- Minimax recursively computes values by backing up from terminal states (or a search-depth cutoff, using an evaluation function) toward the root, so the value assigned to each node reflects optimal play by both sides from that point onward.
- This models the real-time strategy game well because it captures the adversarial nature (the human is actively working against the AI's objective) and gives the AI a principled way to choose moves that are robust against the opponent's best possible counter-play — not just moves that look good in isolation.

ii. Analyzing the Computational Challenges of Large Search Trees

[3 Marks]

- Exponential growth: the size of the game tree grows exponentially with search depth — roughly b^d nodes, where b is the branching factor (number of legal moves per turn) and d is the search depth; in a real-time strategy game, b can be very large (many possible unit actions), making full-depth Minimax computationally infeasible.
- Time constraints: the AI must decide within strict real-time limits (often a fraction of a second per decision), but plain Minimax explores the entire tree to a given depth, which may not finish in time for complex game states.
- Memory usage: storing/expanding a very large tree (even if traversed depth-first with limited memory) still requires substantial computation to evaluate every branch, which competes with the game engine's own real-time resource needs.
- Diminishing returns vs depth: to look further ahead (better strategic quality) the AI must search deeper, but every extra ply multiplies the cost by the branching factor — creating a direct tension between decision quality and real-time responsiveness.

iii. Explaining How Alpha-Beta Pruning Improves Efficiency

[3 Marks]

Alpha-Beta pruning is an optimization of Minimax that eliminates branches of the game tree that cannot possibly influence the final decision, without changing the result Minimax would have produced.

- It maintains two values while traversing the tree: alpha (the best value the maximizer can guarantee so far) and beta (the best value the minimizer can guarantee so far).
- Whenever a node's value falls outside the current $[\alpha, \beta]$ window (i.e., $\alpha \geq \beta$ at that point), the remaining sibling branches at that node are pruned/skipped, because a rational opponent would never allow play to reach that branch.
- This does not sacrifice correctness — Alpha-Beta pruning is guaranteed to compute exactly the same move as full Minimax — but it can, with good move ordering, reduce the effective branching factor from b to roughly $\sqrt{b}$, allowing the AI to search roughly twice as deep in the same amount of time.
- In a real-time strategy game with strict decision deadlines, this efficiency gain is what makes deeper (and therefore stronger) lookahead feasible within the available time budget.

iv. Recommending a Strategy for Balancing Optimality and Real-Time Performance

[3 Marks]

Recommended strategy: Alpha-Beta pruning combined with iterative deepening, strong move ordering, and a well-tuned heuristic evaluation function, rather than pursuing full-depth Minimax.

- Iterative Deepening: run Minimax/Alpha-Beta at increasing depth limits (1, 2, 3, ...) within the time budget, always keeping the best move found so far — this guarantees the AI always has a legal, reasonably good move ready even if it is interrupted by the real-time deadline.
- Move ordering: evaluate the most promising moves first (e.g., using a quick heuristic or previous iteration's best move) to maximize the number of branches pruned by Alpha-Beta, since pruning effectiveness depends heavily on the order moves are considered.
- Depth-limited search with a heuristic evaluation function: since reaching true terminal (game-over) states is infeasible in real time, cut off search at a manageable depth and use a domain-specific evaluation function (e.g., resource counts, unit strength, map control) to estimate the value of non-terminal states.
- Additional engineering techniques: transposition tables (caching previously evaluated states reached via different move orders) and quiescence search (extending search slightly in highly volatile/tactical positions) further improve both speed and decision quality.

This combination balances optimality and real-time performance because Alpha-Beta pruning preserves Minimax's decision-theoretic guarantees while discarding irrelevant computation, iterative deepening respects the hard time limit by always returning an anytime-valid answer, and a good evaluation function/move ordering ensures the limited search depth achievable in real time still produces strategically strong decisions.

— End of Assignment —